

LeadOS Website Requirements Document for Kanishka

1. Purpose

Kanishka will build two websites:

1. LeadOS SaaS Website
2. Dummy Client Website

The **LeadOS SaaS Website** is the public product website. It should make LeadOS look like a real enterprise SaaS product.

The **Dummy Client Website** is the demo lead source. It represents a fictional financial partner program, not Rupeezy directly, to avoid IP and branding issues. This site is where a lead fills the form, chooses language and channel, then enters the AI chat or voice pipeline.

For now, both websites should use static/mock data where needed. Dev will integrate real backend APIs later.

2. Tech Stack

Use the standard frontend stack:

```
Vite React
TypeScript
TanStack Query
TanStack Router
Zustand
React Hook Form + Zod
Tailwind CSS
shadcn/ui
lucide-react
```

Optional libraries:

```
Framer Motion for small animations
Recharts only if charts are needed
React Markdown if showing rich text content
```

Do not overbuild animations. The sites should feel clean, fast, and polished.

3. Development Rule

Build the full UI first with mock service functions.

Rules:

- Do not hardcode data directly inside UI components
- Create mock data files
- Create service functions for future API integration
- Use TanStack Query where backend data is expected
- Use React Hook Form + Zod for forms
- Keep loading, empty, and error states ready
- Keep components reusable

Suggested structure:

```
src/  
  routes/  
  components/  
    common/  
    marketing/  
    demo/  
    chat/  
  features/  
    marketing/  
    demo-client/  
    lead-form/  
    chat-widget/  
  services/  
    publicApi.ts  
  mocks/  
    public.mock.ts  
  store/  
    leadSessionStore.ts  
    uiStore.ts  
  types/  
    public.types.ts
```

4. Website 1: LeadOS SaaS Website

4.1 Purpose

The SaaS website is the public face of LeadOS.

It should explain:

- What LeadOS does
- Why sales teams need it
- How AI voice and chat improve lead conversion
- How the product works
- What features are included
- How to access the demo

It should feel like a startup SaaS landing page, not an admin dashboard.

4.2 Route Structure

Use TanStack Router.

Suggested routes:

```
/
/demo
/login
```

or:

```
/
/company
/demo
```

The main route can include all landing sections on one page.

4.3 Primary Navigation

Header nav should include:

```
Product
How It Works
Features
Architecture
Demo
Login
```

Login dropdown or buttons:

```
Admin Login
RM Login
```

CTA buttons:

```
Try Demo
View Admin Portal
View RM Portal
```

4.4 Visual Direction

The SaaS website should feel:

```
Modern
Clean
Enterprise SaaS
Trustworthy
```

AI enabled but not gimmicky
Fast and polished

Use:

White or very light background
Blue / teal accent colors
Large hero headline
Soft cards
Rounded sections
Subtle shadows
Clean icons
Simple micro interactions

Avoid:

Too much text
Dark cyberpunk styling
Too many animations
Random AI buzzwords
Rupeezy branding

4.5 Main Sections

Section 1: Hero

Purpose:

Immediately explain the product.

Suggested headline:

Convert raw leads into qualified sales opportunities using multilingual AI voice and chat.

Suggested subtext:

LeadOS helps sales teams contact leads faster, handle objections consistently, score intent, and hand off high quality opportunities to human relationship managers.

CTA buttons:

Try Demo
View Product Flow

Secondary links:

Admin Portal
RM Portal

Hero visual idea:

Lead → AI Conversation → Score → RM Handoff

Can be shown as simple connected cards.

Section 2: Problem

Show the core problem.

Cards:

Leads go cold after delays
Language mismatch reduces trust
RMs cannot handle campaign spikes
Handoffs lack conversation context

Each card should have:

Icon
Short title
One sentence description

Section 3: Solution

Explain LeadOS simply.

Suggested copy:

LeadOS acts as an AI lead conversion layer between campaign leads and sales teams. It starts conversations, detects language, follows the approved sales script, handles common objections, scores the lead, and routes the next action automatically.

Solution flow:

Lead Capture → Voice / Chat AI → Objection Handling → Scoring → RM Handoff

Section 4: How It Works

Use a step based section.

Steps:

1. Capture leads from forms, campaigns, or CSV
2. AI engages through voice or chat
3. Agent handles language and objections
4. Transcript is scored as Hot, Warm, or Cold

5. RMs receive Hot leads with full context
6. Warm leads get WhatsApp follow-up links

Each step should be concise.

Section 5: Features

Feature cards:

Multilingual voice and chat
Structured sales workflow
Five objection handling
Explainable Hot/Warm/Cold scoring
RM handoff with transcript and next action
WhatsApp follow-up link generation
Admin dashboard analytics
RAG-ready knowledge base

Do not over-explain. One line per feature is enough.

Section 6: Product Screens Preview

Show three product areas:

Dummy Client Website
Admin Portal
RM Portal

Each card should include:

Short description
Button/link

Example:

Admin Portal: Monitor funnel, scores, conversations, users, and follow-ups.
RM Portal: See assigned Hot leads with score reason, transcript, objections, and next action.
Demo Client Website: Submit a lead and interact with the AI.

Section 7: Architecture Preview

Show the HLD image or a simple architecture card.

Content:

LeadOS connects lead capture, AI conversation, scoring, routing, dashboards, and follow-up workflows.

Can include:

Vapi
OpenAI
LangGraph
LangChain
Mem0
Supabase
n8n

Keep this section simple. Do not make it too technical on the marketing page.

Section 8: Demo CTA

Final strong CTA.

Buttons:

Open Demo Client Website
Open Admin Portal
Open RM Portal

Footer

Footer should include:

LeadOS
AI Lead Conversion Platform
Hackathon MVP
Team name optional

4.6 SaaS Website Components

Build reusable components:

MarketingLayout
Navbar
HeroSection
ProblemCards
SolutionFlow
HowItWorksSteps
FeatureGrid
ProductPreviewCards
ArchitecturePreview
CTASection
Footer

4.7 SaaS Website Mock Data

Create a mock data file for:

features
problemCards
howItWorksSteps
productPreviewCards
techStackItems

4.8 SaaS Website Success Criteria

The SaaS website is complete if:

It looks like a real product landing page
It explains the problem and solution clearly
It links to the dummy client demo
It links to Admin and RM portals
It avoids Rupeezy branding
It has a polished hero and feature sections
It is responsive enough for laptop demo

5. Website 2: Dummy Client Website

5.1 Purpose

The Dummy Client Website simulates a real customer's lead capture page.

It should represent a fictional financial partner program. It should not use Rupeezy branding.

Possible names:

FinPartner Pro
PartnerGrowth Finance
Demo Finance Partner Program

Recommended name:

FinPartner Pro

The purpose is to prove:

Lead submits form → lead enters backend → AI chat or voice begins →
transcript/scoring/routing happens

This website does not need to be as polished as the SaaS website, but it must be clean and usable.

5.2 Route Structure

Suggested routes:

```
/demo-client  
/demo-client/thank-you  
/demo-client/chat/:leadId
```

or if separate app:

```
/  
/thank-you  
/chat/:leadId
```

5.3 Visual Direction

The dummy client website should feel like a simple finance partner landing page.

Use:

```
Clean white background  
Finance style cards  
Trustworthy blue/green accents  
Simple form layout  
Clear CTA  
Minimal animations
```

Avoid:

```
Rupeezy name  
Rupeezy logo  
Real financial claims we cannot prove  
Too much polish that wastes time
```

5.4 Main Sections

Section 1: Dummy Partner Program Hero

Suggested headline:

```
Grow your income as a finance partner.
```

Suggested subtext:

```
Join a demo partner program built for advisors, referrers, and finance  
professionals. Submit your details and our AI assistant will help you  
understand the opportunity.
```

CTA:

```
Talk to AI Assistant
```

Section 2: Benefits

Show fake/demo benefits similar to theme, but generic.

Cards:

Zero joining fee
High revenue share
Fast payouts
Partner support

Do not use exact Rupeezy branding. Generic wording is fine.

Section 3: Lead Form

This is the most important part.

Fields:

Name
Phone
Email optional
City
Role
Approx network size
Preferred language
Preferred channel
WhatsApp consent

Role options:

Financial Advisor
MFD
Insurance Agent
Finance Influencer
Referrer
Other

Preferred language options:

English
Hindi
Hinglish

Preferred channel options:

Chat
Voice

WhatsApp consent checkbox text:

I agree to receive follow-up information through WhatsApp.

Submit button:

Start Conversation

or:

Submit and Talk to AI

5.5 Lead Form Validation

Use React Hook Form + Zod.

Validation rules:

Name required
Phone required
Phone must be valid length
Email optional but must be valid if provided
City optional
Role required
Preferred language required
Preferred channel required
WhatsApp consent optional but should be captured

Suggested Zod fields:

```
type LeadFormValues = {  
  name: string;  
  phone: string;  
  email?: string;  
  city?: string;  
  role: string;  
  networkSize?: string;  
  preferredLanguage: "english" | "hindi" | "hinglish";  
  preferredChannel: "chat" | "voice";  
  whatsappConsent: boolean;  
};
```

5.6 Lead Submit Flow

When the form is submitted:

Form validates → createPublicLead() called → backend returns leadId and leadSessionToken → if preferredChannel = chat, open chat page/widget → if preferredChannel = voice, show request call / start voice state

Mock service:

```
createPublicLead(payload)
```

Later real API:

POST /public/leads

Request:

```
{
  "clientSlug": "finpartner-pro",
  "campaignId": "campaign_1",
  "name": "Amit Sharma",
  "phone": "9876543210",
  "email": "amit@example.com",
  "city": "Hyderabad",
  "role": "financial_advisor",
  "networkSize": "25 to 30",
  "preferredLanguage": "hinglish",
  "preferredChannel": "chat",
  "whatsappConsent": true,
  "source": "website"
}
```

Response:

```
{
  "success": true,
  "data": {
    "lead": {
      "id": "lead_1",
      "name": "Amit Sharma",
      "status": "pending_contact",
      "preferredLanguage": "hinglish",
      "preferredChannel": "chat"
    },
    "leadSessionToken": "temporary_public_token",
    "nextAction": "start_chat"
  }
}
```

5.7 Chat Flow

If preferred channel is chat:

Lead submits form → chat opens → startChat() called → AI greeting appears → user sends messages → assistant replies → user ends chat or conversation completes → score runs later

For static phase:

Use mock assistant replies.

Mock flow should include at least one objection example:

User:

I already have another broker.

Assistant:

That is actually a good sign because you already understand this business. Are you getting full revenue share and fast payouts with your current setup?

Chat UI Components

```
ChatWindow
ChatHeader
ChatMessageBubble
ChatInput
TypingIndicator
EndConversationButton
ConversationCompletedCard
```

Chat Page Should Show

```
Lead name
Selected language
Message history
Input box
Send button
End conversation button
```

Mock services

```
startChat(payload)
sendChatMessage(payload)
endChat(payload)
```

Later real APIs:

```
POST /chat/start
POST /chat/message
POST /chat/end
```

5.8 Chat API Expected Responses

startChat response

```
{
  "success": true,
  "data": {
    "conversation": {
      "id": "conv_1",
      "leadId": "lead_1",
```

```

        "channel": "chat",
        "status": "started",
        "language": "hinglish"
    },
    "assistantMessage": {
        "role": "assistant",
        "content": "Hi Amit, I can explain the partner program in Hinglish. Shall I quickly tell you how it works?"
    }
}

```

sendChatMessage response

```

{
  "success": true,
  "data": {
    "conversationId": "conv_1",
    "assistantMessage": {
      "role": "assistant",
      "content": "That is a fair concern. You can start with a small serious network and grow gradually."
    },
    "conversationState": {
      "stage": "objection_handling",
      "language": "hinglish"
    }
  }
}

```

endChat response

```

{
  "success": true,
  "data": {
    "conversation": {
      "id": "conv_1",
      "status": "completed"
    },
    "score": {
      "classification": "hot",
      "totalScore": 82,
      "reason": "Asked for onboarding details and agreed to WhatsApp follow up."
    },
    "routing": {
      "type": "rm_handoff",
      "status": "assigned_to_rm"
    }
  }
}

```

For static phase, show the final score card only if useful. On the lead-facing site, we can simply show:

Thanks. Our team will follow up with the next steps.

Do not expose all internal scoring to the lead unless needed.

5.9 Voice Flow

If preferred channel is voice:

Lead submits form → lead created → voice request screen shown →
startVoiceSession() called if ready → voice session starts or request call
confirmation shown

For MVP, keep UI flexible.

Voice UI states

Requesting call
Voice session starting
Call in progress
Call completed
Fallback to chat

Buttons

Start Voice Call
Use Chat Instead
Back to Form

Mock service:

```
startVoiceSession(payload)
```

Later real API:

```
POST /voice/start
```

Expected response:

```
{
  "success": true,
  "data": {
    "conversation": {
      "id": "conv_2",
      "channel": "voice",
      "status": "started"
    },
    "voice": {
      "provider": "vapi",
      "sessionId": "vapi_call_id",
      "joinUrl": "https://example.com/vapi-session"
    }
  }
}
```

```
}  
}  
}
```

If Dev provides a Vapi widget or join URL, open it.

If not ready, show:

Your call request has been received. For the live demo, you can continue with chat.

5.10 Thank You / Completion State

After chat or voice completes, show a simple thank you message.

Suggested text:

Thanks for speaking with our AI assistant. If you are a good fit, a relationship manager will follow up with the next steps.

If WhatsApp consent was given:

We may also send follow-up details on WhatsApp.

Do not show internal Hot/Warm/Cold classification to lead by default.

5.11 Dummy Client Website Components

Build reusable components:

```
DemoClientLayout  
DemoHero  
BenefitCards  
LeadForm  
LanguageSelector  
PreferredChannelSelector  
ConsentCheckbox  
ChatWindow  
ChatMessageBubble  
TypingIndicator  
VoiceRequestCard  
ThankYouCard
```

5.12 Dummy Client Mock Data

Create mock data for:

```
benefits  
roles
```

languages
network size options
mock chat replies
mock lead response

Minimum mock chat states:

Greeting
Benefit pitch
Existing broker objection
Not enough contacts objection
Closing / follow up

6. Shared Public Services

Create service file:

src/services/publicApi.ts

Functions:

```
getPublicClientConfig(clientSlug)  
createPublicLead(payload)  
startChat(payload)  
sendChatMessage(payload)  
endChat(payload)  
startVoiceSession(payload)
```

7. Public Client Config

Dummy website should load public config.

Mock service:

```
getPublicClientConfig("finpartner-pro")
```

Later real API:

```
GET /public/client/:clientSlug
```

Expected response:

```
{  
  "success": true,  
  "data": {  
    "client": {  
      "id": "client_1",  
      "name": "FinPartner Pro",  
      "slug": "finpartner-pro"  
    },  
  },  
}
```

```
    "defaultCampaign": {
      "id": "campaign_1",
      "name": "Demo Partner Campaign"
    },
    "supportedLanguages": ["english", "hindi", "hinglish"],
    "supportedChannels": ["chat", "voice"]
  }
}
```

8. Zustand Store

Use Zustand for lightweight local state only.

Store:

```
leadId
leadSessionToken
currentConversationId
selectedLanguage
selectedChannel
chatOpen state
voiceSession state
```

Do not store server data in Zustand when TanStack Query is better.

9. TanStack Query Hooks

Suggested hooks:

```
usePublicClientConfig(clientSlug)
useCreatePublicLead()
useStartChat()
useSendChatMessage()
useEndChat()
useStartVoiceSession()
```

Use mutations for form submit and chat actions.

10. Loading, Empty, and Error States

Both websites should handle:

```
Form submitting
Lead creation failed
Chat loading
AI typing
Voice session starting
Voice unavailable
Conversation completed
```

Error examples:

Could not create lead. Please try again.
Could not start chat. Please try again.
Voice is unavailable right now. Continue with chat.

11. Responsive Requirements

Both websites should work well on:

Laptop
Tablet size if possible
Mobile basic support

Hackathon demo will likely be laptop, so laptop layout is priority.

12. Most Important Demo Path

Kanishka's work is complete for demo if this works smoothly:

Open LeadOS SaaS website → click Try Demo → open Dummy Client Website → fill lead form → select Hinglish → choose Chat → submit → chat opens → lead raises objection → AI replies → conversation completes → thank you state appears

This path must be polished.

13. Build Priority

Priority 1

Dummy Client Website
Lead form
Validation
Create lead mock service
Chat UI
Mock chat flow
Thank you state
CTA links to Admin and RM portals

Priority 2

LeadOS SaaS Website
Hero
Problem section
How it works
Features
Demo CTA
Admin/RM login links

Priority 3

Voice request UI
Voice session state
Architecture preview
Extra animations
Responsive polish

14. Final Success Criteria

Kanishka's work is complete for MVP if:

LeadOS SaaS Website looks like a real SaaS landing page
SaaS website links to dummy client, Admin Portal, and RM Portal
Dummy Client Website has working lead form with validation
Lead form captures language, channel, role, phone, and WhatsApp consent
Chat UI works with static/mock replies
Chat supports at least one objection example
Voice request UI exists even if real call integration comes later
Thank you state appears after conversation
All data access is through service functions ready for API integration
No Rupeezy branding is used